

Toronto Metropolitan University
Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Distributed Cloud Computing
Course Number:	COE892
Semester/Year:	W2026
Instructor:	Dr. Muhammad Jaseemuddin
Assignment/Lab Number:	Project
Assignment/Lab Title:	Project Proposal
Submission Date:	9 March 2026
Due Date:	9 March 2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Khan	Muhammad	501055504	8	M.K
Hassan	Shaheer	501053803	8	S.H
Arif	Abdullah	501122593	8	A.A
Gupta	Abhinav	501089832	8	A.G

**By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct.*

Abstract

This project proposes a distributed air traffic control (ATC) simulation built using a microservices architecture. Traditional ATC systems are often centralized, which can limit scalability and introduce single points of failure. To address this, the system decomposes ATC functionality into independent services that communicate through an event-driven messaging system. RabbitMQ is used as a message broker for asynchronous communication, Redis maintains shared real-time state such as aircraft positions and sector ownership, and PostgreSQL stores system events for logging and analysis. All components are containerized using Docker to ensure consistent deployment and reproducibility. At the time of this interim report, the shared infrastructure, messaging framework, Radar Simulation Service, and Sector Control Services have been implemented. Remaining components, including the Runway and Ground Control Service, Conflict Detection Service, WebSocket Gateway, real-time dashboard, and centralized Logging Service, will be developed in the next phase to complete the distributed ATC simulation.

Introduction

Problem and Motivation

In real life, air traffic control has to track many aircraft at once, coordinate runways, and avoid conflicts, all in real time. Classic ATC systems tend to be centralized and monolithic. That means one failure can affect everything, and scaling up is hard. By turning this into a set of smaller services that communicate over messages, we get a more resilient and scalable setup. It also lets us apply the distributed and cloud computing ideas from the course in a concrete way.

Project Goals

We want to: (1) split ATC into microservices (radar, sectors, runway, conflict, gateway, logging) so each part can be developed and scaled on its own; (2) use RabbitMQ so services communicate asynchronously and stay loosely coupled; (3) keep shared, up-to-date state (positions, ownership, runway queues) in Redis; (4) store all events in PostgreSQL for auditing and analysis; and (5) provide a real-time web dashboard over WebSockets so we can see aircraft, sectors, runways, and conflict alerts live. These goals match what we're learning in COE892 about event-driven systems, messaging, and shared state.

Approach

We're building a multi-airport-style simulation around Toronto Pearson (YYZ). The Radar produces position updates for simulated aircraft. Sector services take ownership of aircraft in their airspace and hand them off when they cross sector boundaries. Later we'll add a Runway service to assign runways via a queue, a Conflict service to detect separation violations, a Gateway to push events to the dashboard over WebSockets, and a Logging service to write

everything to Postgres. Everything runs in Docker and uses one common event format and a single topic exchange. In this interim report we only describe the design and the part we've actually built: the shared foundation plus Radar and Sectors. The rest is planned for the next phase.

System Design

High-Level Architecture

We use one RabbitMQ topic exchange, `atc.events`. Every message has the same envelope: `event_id`, `timestamp`, `type` (same as the routing key), `aircraft_id`, `source_service`, and a data payload. Routing keys we use (or will use) include `aircraft.position`, `aircraft.handoff.request`, `aircraft.handoff.accepted`, `runway.request`, `runway.assigned`, and `conflict.alert`. Radar publishes positions. Sector A and Sector B consume them, update Redis with positions and who owns each aircraft, and publish handoff requests when an aircraft leaves their sector. Once we add it, the Runway service will listen for runway requests and publish assignments; Conflict will listen for positions and publish alerts when separation is broken; the Gateway will forward the right events to the browser; and Logging will subscribe to everything and store it in Postgres. No service talks to another directly, they only publish and subscribe. That keeps things decoupled and easier to scale.

Infrastructure and Shared Components

RabbitMQ hosts the topic exchange. We use durable queues and retry logic on connect so services can recover after a restart. Redis holds the live state: positions under `pos:{id}`, ownership under `owner:{id}`, and later we'll use it for clearances and runway queues. We give keys TTLs so old data expires. Postgres will hold an `event_log` table (`event_type`, `aircraft_id`, `source_service`, `timestamp`, `payload`) and a `flight_plans` table for later. Docker Compose wires up RabbitMQ, Redis, Postgres, and our app services with health checks and shared env vars (`AMQP_URL`, `REDIS_URL`, `POSTGRES_DSN`, `LOG_LEVEL`). We also have a shared Python package that defines the event model (we use Pydantic), the routing key constants, helpers for publishing and subscribing on RabbitMQ, Redis helpers (JSON get/set, locks, queues), Postgres helpers, and logging. That way every service uses the same event shape and the same way of talking to the broker and Redis.

Design of the Radar and Sector Services (Implemented)

The Radar's job is to produce position updates at a fixed interval (we use 2 seconds). We simulate a small number of aircraft, for YYZ we have two: one departure and one arrival. Each aircraft has a phase (e.g. taxi, takeoff_roll, climbing, departed for the departure; holding, approach, final, landed for the arrival). Positions come from waypoint paths: lists of (lat, lon, altitude, speed, ticks) keyed by runway so we can plug in different paths for takeoff vs approach. Once the Runway service and the clearance flow exist, the Radar will read clearance and

runway from Redis and move aircraft through their phases (e.g. holding → approach when cleared to land). For now it just publishes `aircraft.position` with whatever phase and position we have.

The two Sector services share the same code; we only change the env var `SECTOR_ID` (`SECTOR_A` vs `SECTOR_B`). Each sector has a bounding box: Sector A from longitude -79.7 to -79.45, Sector B from -79.45 to -79.2 (same lat range, 43.5 to 43.8). When we get a position we update Redis and, if the position is inside our sector, we set ourselves as owner. If the aircraft has crossed into the other sector we publish `aircraft.handoff.request`. The other sector's worker gets it and publishes `aircraft.handoff.accepted` and takes over ownership in Redis. Later we'll add logic so that when an aircraft needs a runway (e.g. in holding or on taxi) we publish `runway.request`, and when we get `runway.assigned` we write the clearance and runway id to Redis so the Radar can advance the aircraft.

Design of Remaining Services (To Be Implemented)

Runway will subscribe to `runway.request`, push aircraft onto a Redis list per airport, and run a background loop that pops one aircraft at a time, picks a runway that doesn't conflict with others in use (we'll use conflict groups, e.g. 06L and 06R can be used together but not with crossing runways), takes a Redis lock on that runway, and publishes `runway.assigned`. Conflict will subscribe to `aircraft.position`, keep the latest position for each aircraft (in memory or Redis), and whenever we get a new position we'll check it against all others; if both horizontal and vertical separation are below the minimums we publish `conflict.alert`. The Gateway will open a WebSocket, subscribe to positions, runway assignments, and conflict alerts, and push them to every connected client; we'll also need a reset endpoint (e.g. `POST /api/reset`) and the actual dashboard page. Logging will bind to the exchange with a wildcard so it gets every event and insert each one into `event_log`. The dashboard will show sectors, runways, aircraft, conflict lines, and a scrollable event list, we're thinking Leaflet for the map.

Partial Implementation

What we've actually built so far is the shared layer, Radar, and the two Sector instances. That's roughly the first third of what we planned.

Shared Library and Infrastructure

In the shared package we have the `ATCEvent` model and the routing keys. The RabbitMQ helpers do connect-with-retry, declare the topic exchange, and expose publish and subscribe. Subscribe takes a queue name, a routing key, and a callback; when a message arrives we parse it into an `ATCEvent` and call the callback. Redis helpers handle JSON serialization, TTLs, locks, and a simple FIFO queue. We have an init script for Postgres that creates `event_log` and `flight_plans` with indexes so Logging can use them when we build it. Docker Compose builds each service from its own Dockerfile and mounts the shared package so everyone gets the same events and helpers.

Radar Simulation Service

The Radar app is FastAPI. On startup we connect to RabbitMQ and Redis and start a background task. The task loops every 2 seconds (configurable). For each aircraft we keep phase and a tick counter; we have waypoint lists for takeoff (e.g. RWY_06L) and approach (e.g. RWY_06R) so we can interpolate lat, lon, altitude, speed. Right now we're not reading clearance from Redis yet because the Runway flow isn't there, so the aircraft don't advance past holding/taxi until we add that. We build an ATCEvent with type aircraft.position and publish it. All the aircraft and waypoint definitions live in the Radar's domain module so the rest of the app stays clean.

Sector Control Services

We run two processes with the same code, one with SECTOR_ID=SECTOR_A and one with SECTOR_ID=SECTOR_B. Each has a worker that subscribes to aircraft.position. On each message we write the position to Redis and check if the aircraft is still inside our sector (min/max lat and lon). If it's inside we set owner:{aircraft_id} to our sector id. If it's outside we figure out which sector it's in and publish aircraft.handoff.request. We also subscribe to handoff request and handoff accepted. When we get a request that's for us we publish accepted and update Redis so we're the owner. We haven't added the runway request or clearance-writing logic yet; that's for when Runway is ready. Boundary checks are in the domain layer using the sector boxes we defined.

Remaining Work

We still need to do the following.

- **Runway & Ground Control:** Implement the consumer for runway.request, the Redis queue and the background processor that assigns runways using conflict groups and a lock, and publish runway.assigned. Then add to Sector the logic to send runway requests when an aircraft needs a runway and to write clearance to Redis when we get an assignment.
- **Conflict Detection:** Implement the consumer for aircraft.position, keep the latest positions (in memory or Redis), apply the separation rules (horizontal and vertical minimums), and publish conflict.alert when two aircraft are too close.
- **Gateway:** WebSocket server, subscribe to positions, runway assignments, and conflict alerts, and broadcast to all connected clients. Plus the reset API and serving the dashboard page.
- **Real-time dashboard:** The actual web page, map with sectors and runways, aircraft markers (with heading and maybe trails), conflict lines, event log, and connection to the Gateway over WebSocket. We plan to use something like Leaflet for the map.
- **Logging:** One consumer that binds with a wildcard to the exchange and inserts every event into the event_log table in Postgres.

- **Tying it together:** Once Runway and Sector clearance are in place, we'll have the Radar read clearance from Redis and move aircraft through their phases (e.g. holding → approach, taxi → takeoff). Then we'll test end-to-end and add things like reconnection and idempotency where needed for the final report.

Team Responsibilities

We split the work like this. We might adjust as we go.

Member	Responsibilities
Muhammad Khan	Radar Simulation Service and aircraft/waypoint models; Docker and Docker Compose; RabbitMQ and shared event/messaging setup.
Shaheer Hassan	Sector Control Services and handoff protocol; Conflict Detection Service; Redis schema and shared state usage.
Abdullah Arif	Runway & Ground Control Service; real-time dashboard (Leaflet, WebSocket client); documentation.
Abhinav Gupta	Gateway Service (WebSocket server, consumer, reset API); Logging Service and PostgreSQL persistence; shared library (events, Redis/Postgres helpers, logging config).

Conclusion

Our goal is a distributed ATC simulation that uses messaging and shared state the way we're learning in COE892. The design is in place: one topic exchange, one event envelope, Redis for live state, Postgres for history. So far, we've implemented the shared infrastructure and the Radar and Sector services, about a third of what we planned. Runway, Conflict, Gateway, Logging, and the dashboard are still to be built. We'll finish those in the next phase and report on them in the final report.

References

- [1] RabbitMQ, "RabbitMQ Documentation," RabbitMQ, 2024. [Online]. Available: <https://www.rabbitmq.com/documentation.html>
- [2] Redis Ltd., "Redis Documentation," Redis, 2024. [Online]. Available: <https://redis.io/docs/>
- [3] Docker Inc., "Docker Documentation," Docker, 2024. [Online]. Available: <https://docs.docker.com/>
- [4] S. Ramírez, "FastAPI Documentation," FastAPI, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>